



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Indian Institute of Technology Hyderabad

EE2801- Digital Signal Processing Lab(DSP LAB)

Experiment -7

Professor: Kiran

Done by

KURRE VINAY (EE23BTECH11036)

QUESTION:

IIR filter

NON-Pipeline code

Listing 1: 4-point Radix-2 FFT Verilog Module

```

module iir_filter (
    input wire clk,
    input wire reset,
    input wire signed [15:0] x,
    output wire signed [15:0] y
);
    parameter NUM_SECTIONS = 47;
    reg signed [15:0] b0 [0:NUM_SECTIONS-1];
    reg signed [15:0] b1 [0:NUM_SECTIONS-1];
    reg signed [15:0] b2 [0:NUM_SECTIONS-1];
    reg signed [15:0] a1 [0:NUM_SECTIONS-1];
    reg signed [15:0] a2 [0:NUM_SECTIONS-1];
    reg signed [15:0] G [0:NUM_SECTIONS];
    reg signed [15:0] G_final;
    reg signed [15:0] w1 [0:NUM_SECTIONS-1];
    reg signed [15:0] w2 [0:NUM_SECTIONS-1];
    reg signed [31:0] w0_temp, y_temp, final_out;
    reg signed [15:0] section_in, section_out;

    integer i;

    // Read hex coefficient files
    initial begin
        $readmemh("b0.txt", b0); if (b0[0] === 16'hxxxx) $display("ERROR: Failed to load b0.txt");
        $readmemh("b1.txt", b1); if (b1[0] === 16'hxxxx) $display("ERROR: Failed to load b1.txt");
        $readmemh("b2.txt", b2); if (b2[0] === 16'hxxxx) $display("ERROR: Failed to load b2.txt");
        $readmemh("a1.txt", a1); if (a1[0] === 16'hxxxx) $display("ERROR: Failed to load a1.txt");
        $readmemh("a2.txt", a2); if (a2[0] === 16'hxxxx) $display("ERROR: Failed to load a2.txt");
    end

    integer g_file, temp;
    initial begin
        g_file = $fopen("g.txt", "r");
        if (!g_file) begin
            $display("ERROR: Cannot open g.txt");
            $finish;
        end

        for (i = 0; i < NUM_SECTIONS; i = i + 1) begin
            if ($fscanf(g_file, "%d\n", temp) != 1) begin
                $display("ERROR: Failed to read gain value at index %0d", i);
                $finish;
            end
            G[i] = temp[15:0];
        end
        if ($fscanf(g_file, "%d\n", temp) != 1) begin
            $display("ERROR: Failed to read final gain value");
            $finish;
        end
    end

```

```

        end
        G_final = temp[15:0];
        $fclose(g_file);
    end
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            for (i = 0; i < NUM_SECTIONS; i = i + 1) begin
                w1[i] <= 0;
                w2[i] <= 0;
            end
            section_out <= 0;
        end else begin
            section_in = x;

            for (i = 0; i < NUM_SECTIONS; i = i + 1) begin
                w0_temp = {section_in, 16'b0} - (a1[i] * w1[i]) - (a2[i] * w2[i]);
                w0_temp = (w0_temp + 32'h00002000) >>> 14;
                y_temp = (b0[i] * w0_temp[15:0]) + (b1[i] * w1[i]) + (b2[i] * w2[i]);
                y_temp = (y_temp + 32'h00002000) >>> 14;

                // Apply section gain
                y_temp = G[i] * y_temp[15:0];
                y_temp = (y_temp + 32'h00002000) >>> 14;
                w2[i] <= w1[i];
                w1[i] <= w0_temp[15:0];

                section_in = y_temp[15:0];
            end
            final_out = G_final * section_in;
            final_out = (final_out + 32'h00002000) >>> 14;

            section_out <= final_out[15:0];
        end
    end

    assign y = section_out;
endmodule

```

Listing 2: Test bench

```

module iir_filter_tb;

    reg clk;
    reg reset;
    reg signed [15:0] x_in;
    wire signed [15:0] y_out;

    integer i, input_file, output_file;
    integer test_case;
    integer temp_in;

    // Instantiate the filter
    iir_filter dut (
        .clk(clk),
        .reset(reset),
        .x(x_in),
        .y(y_out)
    );
endmodule

```

```

);
always #10 clk = ~clk;

initial begin
    clk = 0;
    reset = 1;
    x_in = 0;
    for (test_case = 0; test_case < 3; test_case = test_case + 1) begin
        case (test_case)
            0: begin
                input_file = $fopen("sine_100Hz_q2_14.txt", "r");
                output_file = $fopen("filtered_100Hz_regular.txt", "w");
            end
            1: begin
                input_file = $fopen("sine_1000Hz_q2_14.txt", "r");
                output_file = $fopen("filtered_1000Hz_regular.txt", "w");
            end
            2: begin
                input_file = $fopen("sine_2000Hz_q2_14.txt", "r");
                output_file = $fopen("filtered_2000Hz_regular.txt", "w");
            end
        endcase

        if (input_file == 0) begin
            $display("Error opening input file for test case %d!", test_case);
            $finish;
        end
        reset = 1;
        #20 reset = 0;
        while (!$feof(input_file)) begin
            if ($fscanf(input_file, "%d\n", temp_in) == 1) begin
                x_in = temp_in;

                @(posedge clk);
                $fdisplay(output_file, "%d", y_out);
            end
        end

        $fclose(input_file);
        $fclose(output_file);
        $display("Completed processing for test case %d", test_case);
        #100;
    end

    $display("All test cases processed successfully");
    $finish;
end
endmodule

```

Pipeline code

Listing 3: 4-point Radix-2 FFT Verilog Module

```

module iir_filter_pipeline (

```

```

input wire clk,
input wire reset,
input wire signed [15:0] x,
output wire signed [15:0] y
);

parameter NUM_SECTIONS = 47;

// SOS Coefficients (Q2.14 format)
reg signed [15:0] b0 [0:NUM_SECTIONS-1];
reg signed [15:0] b1 [0:NUM_SECTIONS-1];
reg signed [15:0] b2 [0:NUM_SECTIONS-1];
reg signed [15:0] a1 [0:NUM_SECTIONS-1];
reg signed [15:0] a2 [0:NUM_SECTIONS-1];

reg signed [15:0] G [0:NUM_SECTIONS];
reg signed [15:0] G_final;

reg signed [15:0] w1 [0:NUM_SECTIONS-1];
reg signed [15:0] w2 [0:NUM_SECTIONS-1];

reg signed [15:0] pipeline_data [0:NUM_SECTIONS];

integer i;
initial begin
    $readmemh("b0.txt", b0);
    $readmemh("b1.txt", b1);
    $readmemh("b2.txt", b2);
    $readmemh("a1.txt", a1);
    $readmemh("a2.txt", a2);
end

integer g_file, temp;
initial begin
    g_file = $fopen("g.txt", "r");
    if (!g_file) begin
        $display("ERROR: Cannot open g.txt");
        $finish;
    end
    for (i = 0; i < NUM_SECTIONS; i = i + 1) begin
        if ($fscanf(g_file, "%d\n", temp) != 1) begin
            $display("ERROR: Failed to read gain value at index %0d", i);
            $finish;
        end
        G[i] = temp[15:0];
    end
    if ($fscanf(g_file, "%d\n", temp) != 1) begin
        $display("ERROR: Failed to read final gain value");
        $finish;
    end
    G_final = temp[15:0];
    $fclose(g_file);
end
reg signed [31:0] w0_temp, y_temp; // Declare variables outside of the always block

always @(posedge clk or posedge reset) begin
    if (reset) begin
        for (i = 0; i < NUM_SECTIONS; i = i + 1) begin

```

```

        w1[i] <= 0;
        w2[i] <= 0;
        pipeline_data[i] <= 0;
    end
    pipeline_data[NUM_SECTIONS] <= 0;
end else begin
    pipeline_data[0] <= x;
    for (i = 0; i < NUM_SECTIONS; i = i + 1) begin
        w0_temp = {pipeline_data[i], 16'b0} - a1[i] * w1[i] - a2[i] * w2[i];
        w0_temp = (w0_temp + 32'h00002000) >>> 14;
        y_temp = b0[i] * w0_temp[15:0] + b1[i] * w1[i] + b2[i] * w2[i];
        y_temp = (y_temp + 32'h00002000) >>> 14;
        y_temp = G[i] * y_temp[15:0];
        y_temp = (y_temp + 32'h00002000) >>> 14;
        w2[i] <= w1[i];
        w1[i] <= w0_temp[15:0];

        pipeline_data[i + 1] <= y_temp[15:0]; // Forward to next stage
    end
end
end
wire signed [31:0] final_out = (G_final * pipeline_data[NUM_SECTIONS] + 32'h00002000) >>> 14;
assign y = final_out[15:0];
endmodule

```

Listing 4: Test bench

```

module iir_filter_pipeline_tb;
    reg clk;
    reg reset;
    reg signed [15:0] x_in;
    wire signed [15:0] y_out;
    integer i, input_file, output_file;
    integer test_case;
    integer temp_in;
    iir_filter_pipeline dut (
        .clk(clk),
        .reset(reset),
        .x(x_in),
        .y(y_out)
    );
    always #10 clk = ~clk;
    initial begin
        clk = 0;
        reset = 1;
        x_in = 0;
        for (test_case = 0; test_case < 3; test_case = test_case + 1) begin
            case (test_case)
                0: begin
                    input_file = $fopen("sine_100Hz_q2_14.txt", "r");
                    output_file = $fopen("filtered_100Hz_pipeline.txt", "w");
                end
                1: begin
                    input_file = $fopen("sine_1000Hz_q2_14.txt", "r");
                    output_file = $fopen("filtered_1000Hz_pipeline.txt", "w");
                end
                2: begin

```

```

        input_file = $fopen("sine_2000Hz_q2_14.txt", "r");
        output_file = $fopen("filtered_2000Hz_pipeline.txt", "w");
    end
endcase

if (input_file == 0) begin
    $display("Error opening input file for test case %d!", test_case);
    $finish;
end
reset = 1;
#20 reset = 0;
while (!$feof(input_file)) begin
    if ($fscanf(input_file, "%d\n", temp_in) == 1) begin
        x_in = temp_in;

        @(posedge clk);
        $fdisplay(output_file, "%d", y_out);
    end
end

$fclose(input_file);
$fclose(output_file);
$display("Completed processing for test case %d", test_case);
#100;
end

$display("All test cases processed successfully");
$finish;
end
endmodule

```

Matlab Code

```

load('IIR.mat');
b0 = SOS(:,1);
b1 = SOS(:,2);
b2 = SOS(:,3);
a1 = SOS(:,5);
a2 = SOS(:,6);

% Scale to Q2.14 (multiply by 2^14 and round)
scale = 2^14;
b0_q = round(b0 * scale);
b1_q = round(b1 * scale);
b2_q = round(b2 * scale);
a1_q = round(a1 * scale);
a2_q = round(a2 * scale);
g_q = round(G * scale);
save_binfile('b0.txt', b0_q);
save_binfile('b1.txt', b1_q);
save_binfile('b2.txt', b2_q);

```

```

save_binfile('a1.txt', a1_q);
save_binfile('a2.txt', a2_q);
save_binfile('g.txt', g_q);
Fs= 48000;
duration = 0.1;
t = 0:1/Fs:duration;
freqs = [100, 1000, 2000];
signals = zeros(length(freqs), length(t));

for i = 1:length(freqs)
    signals(i,:) = sin(2*pi*freqs(i)*t);

    % Convert to Q2.14 and clip
    sig_q = round(signals(i,:) * scale);
    sig_q = max(min(sig_q, 32767), -32768); % clip to int16

    % Save as binary strings
    save_binfile(sprintf('sig_%dHz.txt', freqs(i)), sig_q);
end
num_sections = size(SOS,1);
w1 = zeros(num_sections,1);
w2 = zeros(num_sections,1);
matlab_outputs = zeros(length(freqs), length(t));

for sig_idx = 1:length(freqs)
    w1(:) = 0; w2(:) = 0;

    for n = 1:length(t)
        x = signals(sig_idx,n);
        section_in = x;

        for k = 1:num_sections
            w0 = section_in - a1(k)*w1(k) - a2(k)*w2(k);
            y = b0(k)*w0 + b1(k)*w1(k) + b2(k)*w2(k);
            y = G(k) * y;

            w2(k) = w1(k);
            w1(k) = w0;
            section_in = y;
        end

        matlab_outputs(sig_idx,n) = section_in;
    end
end
figure;
for i = 1:length(freqs)

```

```
    subplot(length(freqs),1,i);
    plot(t, matlab_outputs(i,:), 'r');
    title(sprintf('%d MATLAB Filtered ', freqs(i)));
    xlabel('Time (s)');
    ylabel('Amplitude');
    xlim([0 0.1]);
    hold off;
end
function save_binfile(filename, int_array)
    fid = fopen(filename, 'w');
    for i = 1:length(int_array)
        bin_str = dec2bin(typecast(int16(int_array(i)), 'uint16'), 16);
        fprintf(fid, '%s\n', bin_str);
    end
    fclose(fid);
end
```

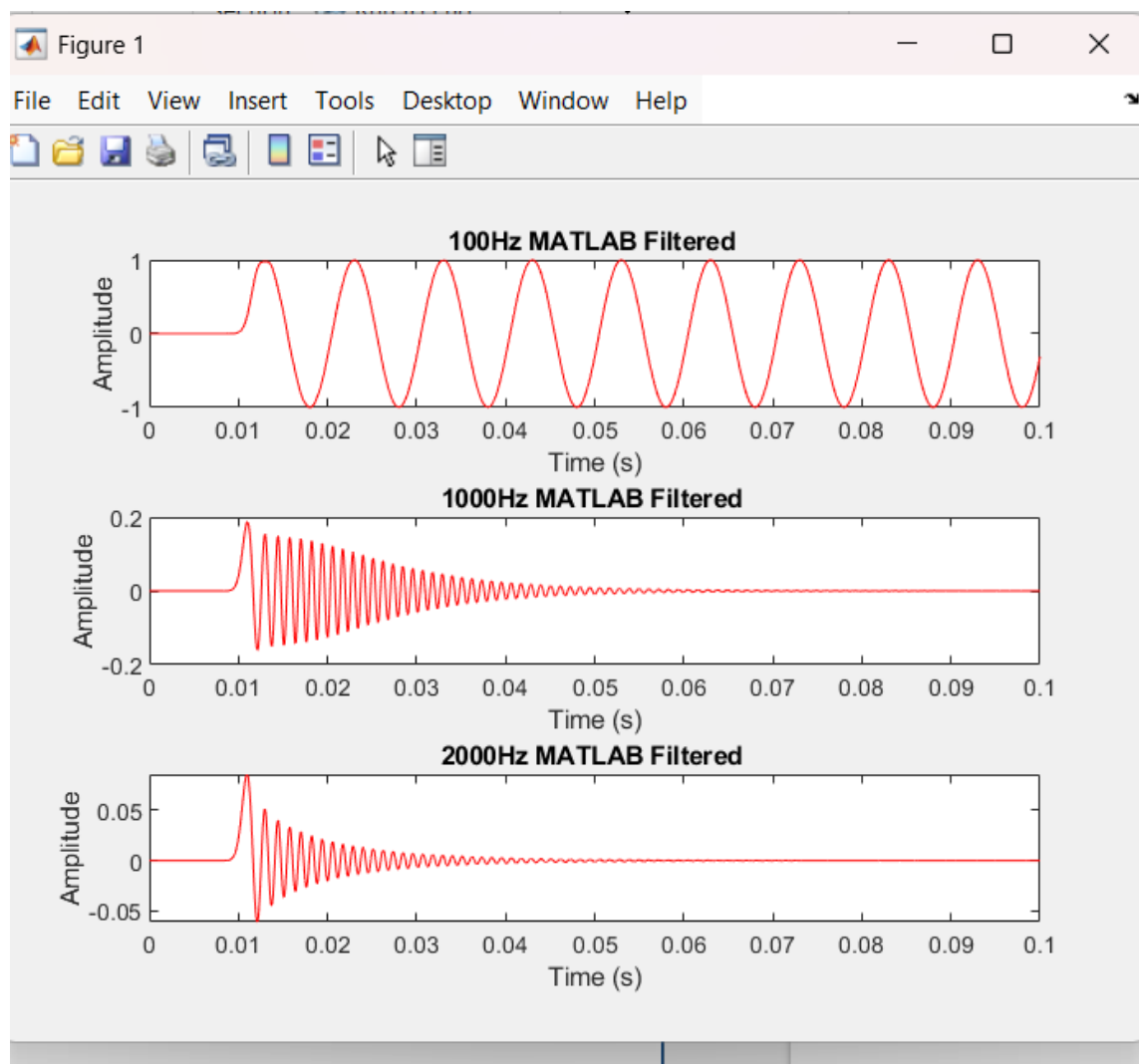


Figure 1: Matlab Output

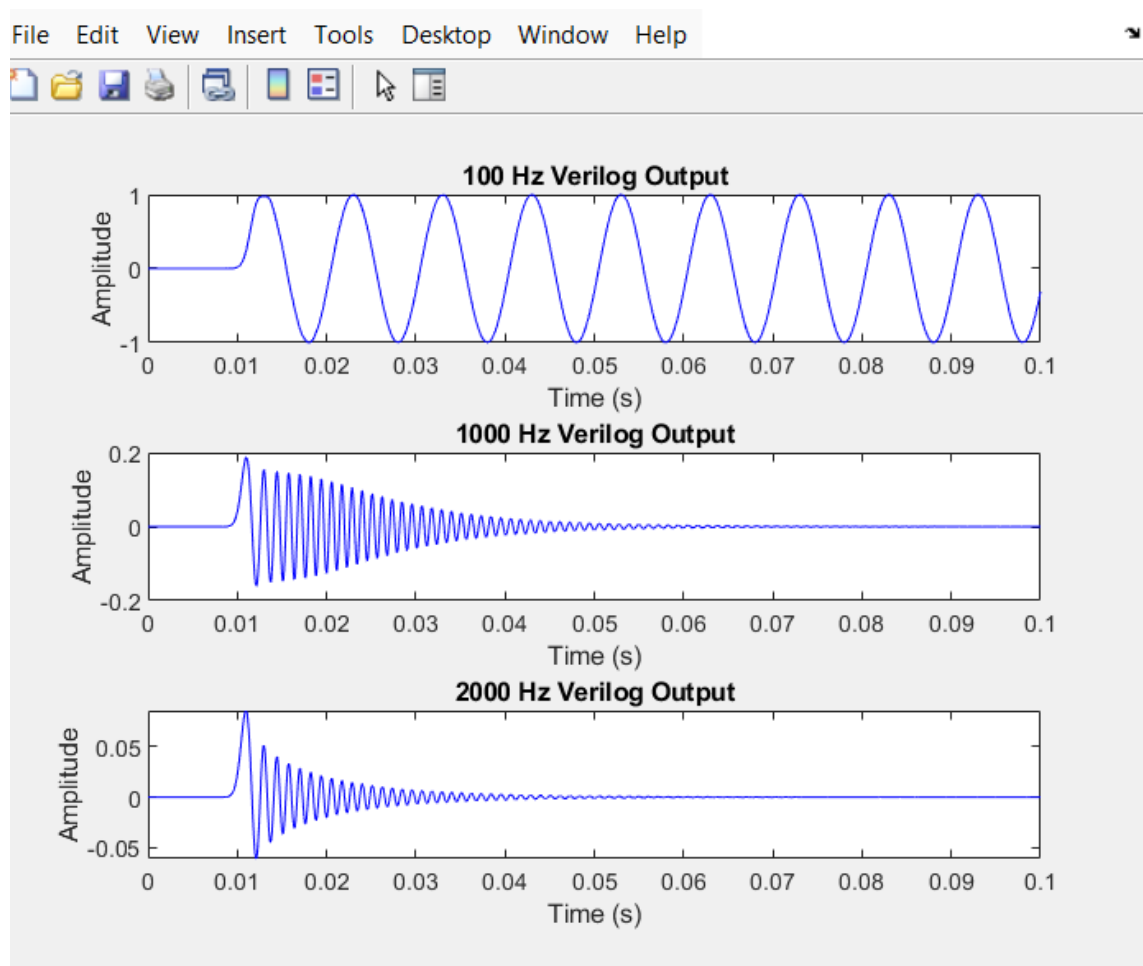


Figure 2: verilog Output

Compartion Code

```
fs = 48000;
freqs = [100, 1000, 2000];
duration_to_plot = 0.1;
max_samples = 0;
figure;
for i = 1:length(freqs)
    freq = freqs(i);
    verilog_file = sprintf('filtered_%dHz_regular.tex', freq);
    matlab_file = sprintf('filtered_%dHz.tex', freq);
    verilog_data = load(verilog_file);
```

```
matlab_data = load(matlab_file);
N = min(length(verilog_data), length(matlab_data));
t = (0:N-1) / fs;
idx_limit = min(N, round(duration_to_plot * fs));
subplot(length(freqs), 1, i);
plot(t(1:idx_limit), verilog_data(1:idx_limit), 'b');
hold on;
plot(t(1:idx_limit), matlab_data(1:idx_limit), 'r');
title(sprintf('%d Hz Verilog Filtered(blue) vs MATLAB Filtered(red)', freq));
xlabel('Time (s)');
ylabel('Amplitude');
legend('Verilog (blue)', 'MATLAB (red)');
grid on;
xlim([0 duration_to_plot]);
end
```

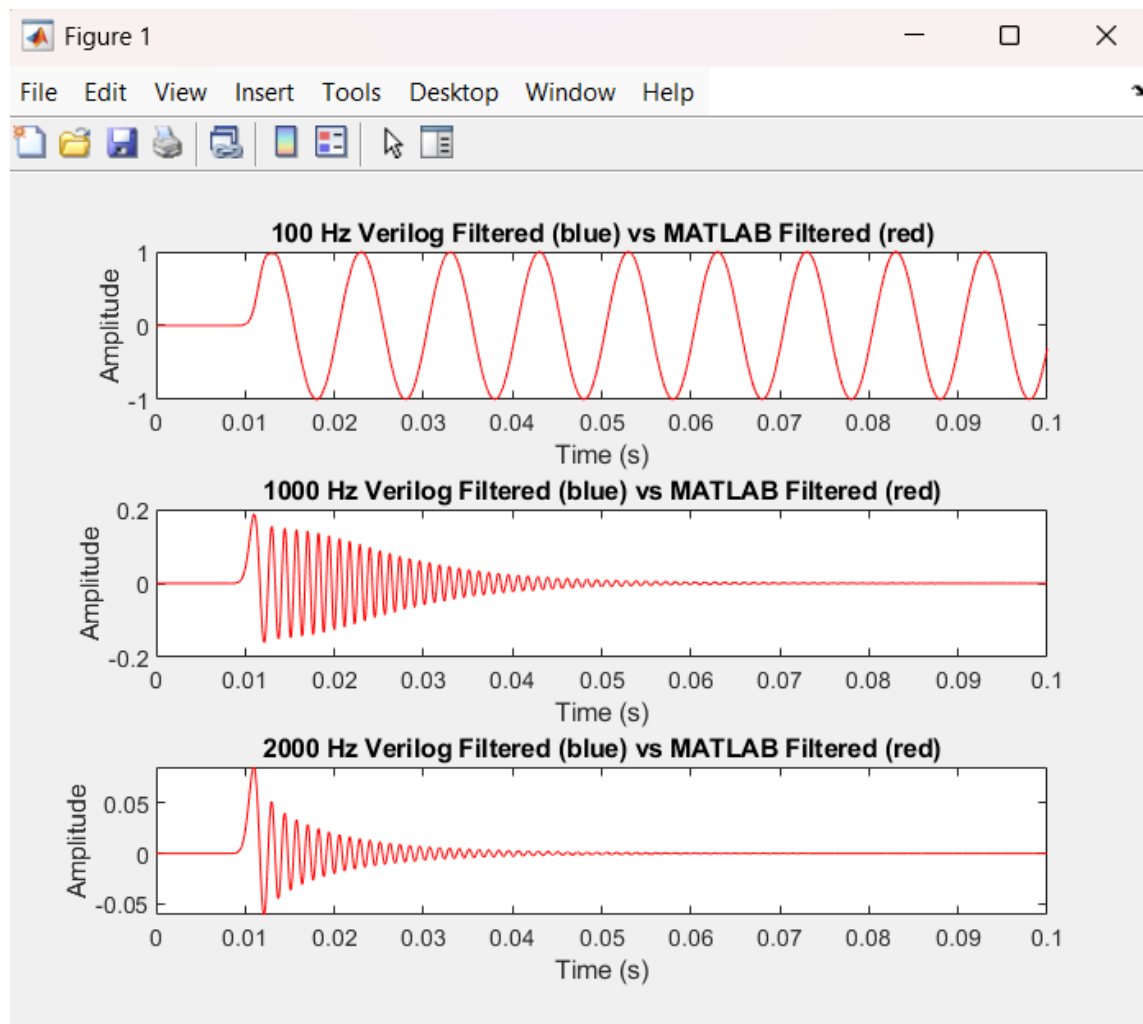


Figure 3: verilog Output vs mathlab output

Conclusion

by seeing the comparison code we can say that the verilog output and mathlab output are exactly matched so the both waves are correct. i had verified using pipelined output, i got same figure